

HybEA: Hybrid Models for Entity Alignment

Nikolaos Fanourakis¹, Fatia Lekbour², Guillaume Renton², Vasilis Efthymiou³,
and Vassilis Christophides^{2,4}

¹ Computer Science Department, University of Crete, Heraklion, Greece
`nikosfanou@csd.uoc.gr`

² ETIS, ENSEA, CYU University, Cergy, France
`{fatia.lekbour,guillaume.renton,vassilis.christophides}@ensea.fr`

³ Depart. of Informatics and Telematics, Harokopio University, Athens, Greece
`vefthym@hua.gr`

⁴ IPAL CNRS IRL 2955, Singapore

Abstract. Entity Alignment (EA) aims to detect descriptions of the same real-world entities among different Knowledge Graphs (KG). Several embedding methods have been proposed to rank potentially matching entities of two KGs according to their similarity in the embedding space. However, existing EA embedding methods are challenged by the diverse levels of structural (i.e., neighborhood entities) and semantic (e.g., entity names and literal property values) heterogeneity exhibited by real-world KGs, especially when they are spanning several domains (DBpedia, Wikidata). Existing methods either focus on one of the two heterogeneity kinds depending on the context (mono- vs multilingual). To address this limitation, we propose a flexible framework called HybEA, that is a hybrid of two models, a novel attention-based factual model, co-trained with a state-of-the-art structural model. Our experimental results demonstrate that HybEA outperforms the state-of-the-art EA systems, achieving a 16% average relative improvement of Hits@1, ranging from 3.6% up to 40% in 5 monolingual datasets, with some datasets that can now be considered as *solved*. HybEA also outperforms state-of-the-art methods in 3 multi-lingual datasets, as well as on 2 datasets that drop the unrealistic, yet widely adopted, one-to-one assumption. Overall, HybEA outperforms all (11) baseline methods in all (3) measures and in all (10) datasets evaluated, with a statistically significant difference.

Keywords: Entity alignment · Knowledge Graphs · Attention

1 Introduction

Knowledge graphs (KGs) are becoming ubiquitous in a wide variety of domains, supporting applications such as question answering [1], entity search [10], and recommender systems [33]. Typically, KGs store machine-readable descriptions of real-world entities, under the form of *structural* information, i.e., relations with neighboring entities, and *factual* information, i.e., names, identifiers, attributes, and literal values. A crucial task when integrating knowledge from

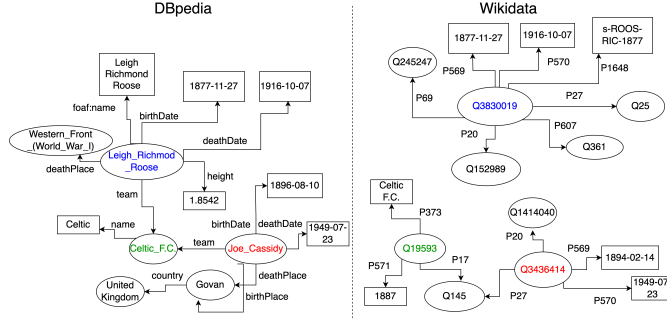


Fig. 1: Motivating example from a real dataset (D-W (S)). Entities (nodes) with the same color are matching entities.

several independently created KGs is to detect whether two nodes refer to the same real-world entity, a task known as *entity alignment (EA)* [15,31,28], entity matching, or entity resolution [6,5,12].

Several KG *embedding* methods have been proposed for EA [17,15,31,20,22], as they can potentially mitigate the symbolic, linguistic, and schematic heterogeneity of independently created KGs. The idea is to represent entities and relations in a low-dimensional embedding space, in which matching entities are expected to be close to each other [42,46]. However, real-world KGs often emphasize different aspects of the same entities. This leads to both structural and semantic heterogeneity [6], making EA a challenging task: two matching entities may have similar factual descriptions but very different neighborhoods, or conversely, similar neighborhoods but inconsistent names or literal values.

Example. Figure 1 illustrates entity alignments of DBpedia with Wikidata. The pair *Leigh_Richmond_Roose/Q3830019* shares similar birth and death dates but no common neighbor (factually homogeneous, structurally heterogeneous), whereas *Joe_Cassidy/Q3436414* have inconsistent factual values but informative neighborhoods (e.g. *Celtic_F.C./Q19593*).

Existing embedding models for EA usually focus mainly on one source of evidence that is not universally reliable. Structure-based methods, such as Knowformer [21], RREA [24], and SelfKG [22], rely mostly on relational neighborhoods, whereas semantic methods, such as COTSAE [40], ZeroEA [17], and BERT-INT [32], exploit names, attributes, or literal values. Such specialization can be effective when the dominant type of heterogeneity is known in advance. However, this assumption is unrealistic in real-world settings, where heterogeneity varies both across datasets and locally across entity pairs.

To address this limitation, we propose HybEA, a hybrid EA framework that jointly exploits factual and structural evidence without assuming in advance which one is more reliable. HybEA combines a novel attention-based factual model with a plug-and-play structural model, co-trained in a semi-supervised manner so that each component enriches the other’s training set across cycles. Compared to co-training introduced in [40], HybEA leverages this approach to jointly cope structural and factual heterogeneity of KGs. In this respect, it

combines two specialized components, one factual and one structural, allowing each model to learn from the modality it is more adapted to. Through reciprocal high-confidence predictions, the two components enrich each other during training and adapt to datasets where different forms of evidence dominate.

In summary, the contributions of this work are the following:

- We analyze ten monolingual and multilingual EA datasets and show that they exhibit different levels of structural and semantic heterogeneity, forming clusters that have only been partially considered in previous evaluations [31,15,18].
- We introduce HybEA a semi-supervised co-training framework combining a novel attention-based factual model with plug-and-play structural backbones. Unlike previous works [40], HybEA considers reciprocal top-1 pseudo-labels from both the structural and factual information of entities while computes attention over the whole attribute vocabulary of each KG.
- We experimentally demonstrate that HybEA outperforms 11 state-of-the-art EA baselines across 10 datasets and 3 evaluation measures, achieving a 16% average relative improvement in Hits@1 on monolingual datasets, with statistically significant gains overall.
- The source code and the datasets used are available: <https://github.com/fanourakis/HybEA>

The remainder of the paper is organized as follows. Section 2 presents our EA framework. Section 3 reports the experimental evaluation against 11 baselines and 10 datasets. Section 4 concludes the paper and discusses future directions.

2 Methodology

According to Appendix B and in order to define an efficient entity alignment model, it is mandatory to consider both structural and factual information, even when this information is strongly heterogeneous between the graphs. In this section, after a brief description of the problem setting, we provide an overview of our approach, followed by details about every component.

2.1 Problem Setting

Following the typical notation [45,39,15], we assume that entities are described in KGs by a collection of triples $\langle h, r, t \rangle$, whose head h is always an entity, and tail t may be either an entity, in which case we call r a *relation* (and $\langle h, r, t \rangle$ a *relation triple*), or a literal (e.g., number, date, string), in which case we call r an *attribute* (and $\langle h, r, t \rangle$ an *attribute triple*).

We represent a knowledge graph as $KG = (E, R, A, L, X, Y)$, where E, R, A, L are sets of entities, relations, attributes, and literals, and $X \subseteq (E \times R \times E)$ and $Y \subseteq (E \times A \times L)$ are the sets of relation and attribute triples, respectively. Given a source $KG_1 = (E_1, R_1, A_1, L_1, X_1, Y_1)$ and a target $KG_2 = (E_2, R_2, A_2, L_2, X_2, Y_2)$, entity alignment seeks the matching pairs

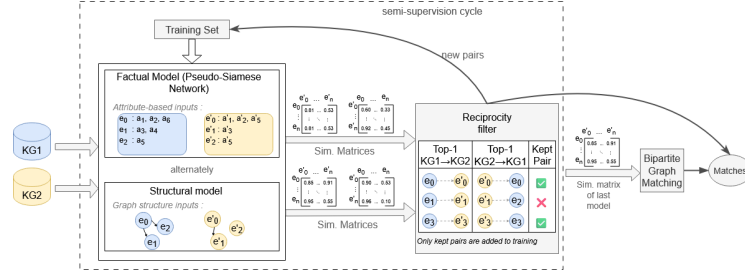


Fig. 2: HybEA architecture.

$M = \{(e_i, e_j) \in E_1 \times E_2 \mid e_i \equiv e_j\}$, where “ $\equiv$ ” denotes the equivalence relationship [45,39]. A seed alignment $S \subseteq M$ and a set $N \subseteq (E_1 \times E_2) \setminus M$ of negative samples may be used for training.

The majority of the related work relies on the conventional one-to-one assumption [30,40,44], where every entity of E_1 matches exactly one entity of E_2 and vice versa, which implies $|M| = |E_1| = |E_2|$. Datasets with partial overlap contain many entities that belong to no alignment pair at all, so that $|M| < \min(|E_1|, |E_2|)$. This is the case of datasets ICEWS-WIKI and ICEWS-YAGO and it is sometimes called the zero-to-one or one-to-zero. Finally, under a many-to-many setting, a single entity may correspond to several entities of the other KG. We evaluate HybEA on datasets of the first two kinds. The reciprocity filter may leave entities unmatched when no mutual top-1 candidate is found. However, this does not constitute an explicit no-match decision, since the final Best Match step (Section 2.6) assigns a candidate to every remaining entity.

2.2 HybEA Overview

Figure 2 presents the high-level architecture of HybEA, involving a *factual component* and a *structural component*. The factual component is implemented as a pseudo-Siamese encoder, with an attention mechanism that gathers matching evidence from attribute triples. The structural component, on the other hand, is responsible for gathering matching evidence originating from structural information of the graph. Both components generate similarity matrices that quantify how likely each entity in the first graph is to correspond to an entity in the second graph, and vice versa. HybEA is constructed around the factual component, into which different structural models can be plugged (Section 2.4).

Our framework is semi-supervised and runs in cycles, where every cycle consists of the co-training of the factual and the structural components. Each component independently identifies a set of *new matching pairs*, and the most confident ones, the reciprocal top-1, are added to the training set for the next component and cycles, as shown in Figure 2. The confidence is determined through a reciprocity filter (detailed in Section 2.5). The execution stops when no new matching pairs are identified by at least one component at the end of a cycle, or when a maximum number of cycles is reached. This process allows both models to gradually refine their predictions across all cycles.

Algorithm 1: HybEA high-level algorithm.

Input: $KG_1 = (E_1, R_1, A_1, L_1, X_1, Y_1)$, $KG_2 = (E_2, R_2, A_2, L_2, X_2, Y_2)$, disjoint subsets of matching pairs $S_{train}, S_{val} \subseteq M$ used as supervision, sets of *unlabeled* candidate entities $U_1 \subseteq E_1, U_2 \subseteq E_2$

Output: Matching pairs $M' \subseteq E_1 \times E_2$

```

1  $M' \leftarrow \emptyset$ ; // set of matching pairs
2  $SMat[E_1][E_2]$ ; // similarity matrix  $E_1 \times E_2$ 
3  $E_1 \leftarrow embed(KG_1)$ ,  $E_2 \leftarrow embed(KG_2)$ ; // embedding init.
4 for  $i$  in  $[1..maxCycles]$  do
    /* train the factual model, update  $SMat$  and  $M'$  */
5    $fm \leftarrow trainFact(S_{train} \cup M', S_{val}, E_1, E_2)$ 
6    $SMat \leftarrow fm.run(U_1, U_2)$ ; // sim. matrix from fm
7    $M_{fm} \leftarrow reciprocity(SMat)$ ;  $M' \leftarrow M' \cup M_{fm}$ ; // new pairs
    /* do the same for the structural model */
8    $sm \leftarrow trainStruct(S_{train} \cup M', S_{val}, E_1, E_2)$ 
9    $SMat \leftarrow sm.run(U_1, U_2)$ ; // sim. matrix from sm
10   $M_{sm} \leftarrow reciprocity(SMat)$ ;  $M' \leftarrow M' \cup M_{sm}$ ; // new pairs
11  $M'_{rest} \leftarrow BipartiteGraphMatching(SMat)$ 
12  $M'' \leftarrow \{(e_i, e_j) \in M'_{rest} \mid \nexists e_k, (e_i, e_k) \in M'\}$ 
13  $M' \leftarrow M' \cup M''$ ; // non-reciprocal matches from last model
14 return  $M'$ 

```

For the remaining entities, not belonging to high-confidence matching pairs, we use the suggestions of a bipartite graph matching algorithm ran on the similarity matrix of the component executed last; more options are discussed in [25,5]. We note that once a positive matching decision is made for a pair, whether it is high-confidence or not, that decision is not retracted, i.e., the number of matches is monotonically increasing. The high-level algorithm of HybEA, without the specific algorithmic decisions on the embedding initialization, the factual and the structural model, as well as the bi-partitie graph matching algorithm, which are all described next in detail, is presented in Algorithm 1.

U_1 and U_2 denote the entities of the two KGs that are not covered by $S_{train} \cup S_{val}$, taken *without* their labels. The test alignment S_{test} is used exclusively for the final evaluation of Algorithm 1. Hence, HybEA is a semi-supervised and transductive alignment framework. In fact, S_{train} and S_{val} provide supervision, while entities in U_1 and U_2 are used without their alignment labels. Although their alignment labels are not used, these entities can influence later cycles through reciprocal pseudo-labeling, while the structural backbones access the complete graphs during training. Once accepted, reciprocal pairs are never removed, so M' can only grow across cycles. The decision rules actually select at most one candidate per source entity, explicit no-match detection and one-to-many or many-to-many alignment are not currently supported.

2.3 Exploiting Factual Information

Let $A_j = (a_1^j, a_2^j, \dots, a_n^j)$ be the totally ordered set of attributes in KG_j . HybEA learns attention weights $\mathcal{A}_j = (a_1^j, a_2^j, \dots, a_n^j)$ over these attributes, from their types and literal values across the entire KG_j , to identify which attributes are most informative for aligning KG_1 and KG_2 .

Let $\mathbf{A} = (\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n)$ be the attribute embeddings of the attributes $A = (a_1, a_2, \dots, a_n)$, and let $\mathbf{V} = (\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_n)$ be the literal (value) embeddings corresponding to the attributes in A , where $v_i \subseteq L$ are literal values. Attention weights are computed by our factual model using the scaled dot-product attention [36], as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1)$$

where Q (query) are the attribute type embeddings of an entity e with m attributes, K (key) corresponds to the attribute type embeddings $\mathbf{A}$ of the entire KG_j with A_j attributes, V (value) corresponds to the literal (values) embeddings of $\mathbf{A}$, and d_k is the dimension of the embeddings, that are used as scaled factor to avoid pushing softmax function to regions where it has extremely small gradients [36].

Having computed the attribute attention weights $\mathcal{A}$, we compute the embedding of an entity e , as

$$\mathbf{e} = \sum_{(e, a_i, u_i) \in Y} a_i \mathbf{u}_i. \quad (2)$$

We employ a pseudo-Siamese network architecture, which has shown good results in [40], to compute different attention weights $\mathcal{A}_1$ and $\mathcal{A}_2$ for KG_1 and KG_2 , respectively, while we compute the literal value embeddings $\mathbf{V}$ of both networks using the same pre-trained sentence embeddings model (i.e., Sentence-BERT [27]).

Finally, to account factual information, we compute the distance of two entities $e_1 \in E_1$ and $e_2 \in E_2$, as the Euclidean distance of their embeddings $d(e_1, e_2) = \|\mathbf{e}_1 - \mathbf{e}_2\|_2$, and we use the contrastive loss function

$$\mathcal{L}_a = (1 - \alpha) \sum_{(e_1, e_2) \in M} d(e_1, e_2) + \alpha \sum_{(e'_1, e'_2) \in N} [\lambda - d(e_1, e_2)]_+, \quad (3)$$

where $0 \leq \alpha \leq 1$ is a hyper-parameter for weighting the negative samples over the positive samples, λ is a margin used for the distance of negative samples (the distance of the negative samples is expected to be larger than λ), and $[\cdot]_+ = \max(0, \cdot)$.

The training and validation of the factual model is abstractly defined in Line 5 of Algorithm 1, using the initial training set S_{train} enriched with any newly discovered pairs M' as training and the initial validation set S_{val} for validation. The newly trained factual model fm returns a similarity matrix $SMat$. The architecture is shown in Figure 3.

2.4 Exploiting Structural Information

HybEA is built around the factual component, while the structural component works as a plug-and-play module, for which different structure-based models

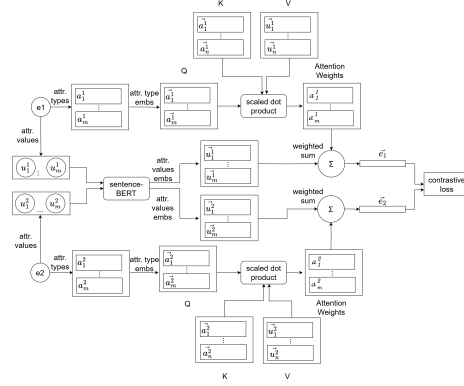


Fig. 3: Factual model architecture.

can be used. To assess this choice, we selected two methods that follow different paradigms, namely Knowformer and RREA, both showing strong results on structural EA datasets. On one hand, Knowformer exploits Transformers over individual relation triples $\langle h, r, t \rangle$. It uses translation-based relational composition to encode positional information and to distinguish the role of head and tail entities, while the attention mechanism captures local relational information. On the other hand, RREA is an attention-based GNN that propagates information through the whole graph. In this way, it provides a more global view of the KG structure. By using relational reflection, defined as an orthogonal transformation, it preserves the spatial structure of entity embeddings, but it distinguishes individual relations less explicitly than Knowformer. The two methods are also implemented in different frameworks, TensorFlow for RREA and PyTorch for Knowformer, which gives us the opportunity to evaluate whether our framework can be used with heterogeneous structural backbones. Throughout the rest of the paper, we denote by HybEA-R and HybEA-K the two instantiations of HybEA that use RREA and Knowformer, respectively, as the structural component. Note that both backbones require as usual the two complete graphs at training time, which is what makes the structural component and hence HybEA as a whole transductive (Section 2.2).

Although both components may perform well in some cases, none is reliable on all datasets: the structural signal helps when factual evidence is too heterogeneous, while factual information helps when the structure is noisy or incomplete. A method relying on a single source is thus likely to fail on part of the datasets, which motivates the hybrid approach presented in next section.

2.5 Semi-supervision and reciprocity filter

HybEA thus alternates between a factual model and a structural model, which are trained independently. In order to inject information into the other model, HybEA leverages a transductive co-training architecture (see the learning protocol described in Section 2.2). To co-train both models, the mutual training

set is incrementally enriched with high-confidence entity pairs identified by a reciprocity filter (Lines 7 and 10 in Algorithm 1).

The employed reciprocity filter [11,24] returns only the entity pairs for which both entities have the other entity as their most probable match. In other words, it keeps only the pairs (e_i, e_j) , with $e_i \in E_1$, $e_j \in E_2$, for which e_j is the top-1 candidate for e_i , and e_i is the top-1 candidate for e_j . These pairs become also part of the final matching pairs on which we evaluate the effectiveness of our system (Line 14 in Algorithm 1) and are not removed in later cycles. For entities not covered by this filter, we return the candidates ranking produced by the last trained component (Alg. 1, L11–13). Finally we also employ an early stopping mechanism as well as a maximum number of semi-supervision cycles (*maxCycles* in Alg. 1).

2.6 Bipartite Graph Matching

In the final step, for entities that have not yet been matched by the reciprocity filter, we employ a typical bipartite matching process. The standard algorithm followed by the majority, if not all, of existing works is the Best Match algorithm [25], i.e., for every node in KG_1 , suggest as its match the most similar entity from KG_2 , according to the Cosine similarity of their embeddings. In HybEA, we employ the embeddings that were generated by the last trained model.

The reciprocity filter returns a partial, injective matching: a pair is returned only if the two entities are mutual top-1 candidates, so entities without a confident counterpart are simply left unmatched. Table 3 shows that the majority of the matches of HybEA are produced by this filter, and that its cumulative precision remains between 0.99 and 1.00 on ICEWS-WIKI and ICEWS-YAGO despite the low overlap of these two datasets. The final Best Match step, in contrast, assigns a candidate to every entity that is still unmatched and therefore implicitly assumes that such a counterpart exists. We keep it only to remain comparable with prior work, whose rank-based measures ($H@k$, MRR) presuppose exactly one ground-truth match per test entity.

A possible extension toward many-to-many alignment would require replacing the current single-match decision rule with a multi-match criterion, such as thresholded similarity scores. Such a rule would be less appropriate for the one-to-one and partial-overlap benchmarks considered here, since their ground truth contains at most one counterpart per entity. The main limitation is experimental: to the best of our knowledge, current EA benchmarks do not provide genuine many-to-many ground truth. Such an extension would therefore require threshold calibration and suitable many-to-many benchmarks. Alternative formulations, such as cluster-to-cluster matching, would additionally require reliable automatic cluster extraction, which remains an open problem.

3 Experimental Evaluation

We evaluate our method on ten EA datasets, chosen in order to cover different cases of structural and factual heterogeneity. We first use **D-W (S)** and **D-**

W (D) [31], which correspond to the sparse and dense versions of DBpedia–Wikidata. These datasets are not only different in terms of KG density. They are also challenging because entity names are often not sufficient to find the correct matches, while many entities do not contain long textual descriptions [15]. Thus, methods that mainly rely on this information, such as KDCoE [2], are not expected to be in their best setting. We also consider **D-W (SRPRS-N)** and **D-W (SRPRS-D)** [16], that keep the same sparse/dense setting, but use SRPRS sampling, giving an entity distribution closer to real-world KGs. **BBC-DB** [13] is a monolingual dataset aligning BBC Music⁵ with the BTC2012 version of DBpedia⁶. Since it contains many attributes for each entity, it allows us to evaluate methods that exploit factual information. We further use **FR-EN**, **JA-EN** and **ZH-EN** [30], three multilingual DBpedia⁷ datasets involving English, French, Japanese and Chinese. Finally, we include **ICEWS-WIKI** and **ICEWS-YAGO**, where ICEWS is aligned with Wikidata and YAGO. These datasets are useful to consider, since they have low KG overlap, strong structural heterogeneity, and relax the total one-to-one coverage assumption by allowing entities with no counterpart in the other KG (Section 2.1).

Experiments were run on a server with 16 AMD EPYC 7232P @ 3.1 GHz cores, 64 GB RAM, an RTX-4090 GPU (24 GB), and Ubuntu 18.04.5. For the structural model of (HybEA-R), we follow RREA [24]: embeddings of size 100, 2 layer architecture with 1 attention head, dropout 0.3, learning rate 0.005, and RMSprop optimizer. For HybEA-K, we adopt Knowformer [21] settings: 12 layers, 4 attention heads, embedding size of 768, various dropouts (input: 0.5, attention: 0.1, feed-forward: 0.3, output: 0.1), residual 0.5, soft label 0.25, addition loss 0.1, learning rate 0.0005, batch size 2048. The factual model uses: $\alpha = 0.8$, $\lambda = 3$, $d_k = 768$, learning rate 0.00005, batch size 24, 1 attention layer per KG (size 768), and input dimensions based on attribute types. For all models, CSLS [19] with $k = 2$ is applied. Early stopping halts training after 3 drops in H@1, with max epochs of 200 (HybEA-K) or 1200 (HybEA-R), and min epochs of 10 (structural) or 5 (factual). Semi-supervision runs up to 4 cycles. The seed alignment is split into 20% train, 10% validation, and 70% test. All reported numbers correspond to a single run, a multi-run variance analysis is left for future work.

Structural-backbone parameters follow their original implementations for comparability, while factual-model hyper-parameters are kept fixed across datasets to avoid dataset-specific tuning. The effect of varying the main design parameters is reported in the appendices: entity initialization, component ordering , and the number of semi-supervision cycles.

To evaluate the benefits of our method, we compare its performance against eleven state-of-the-art baseline methods, namely MTransE [3], Knowformer [21], BERT-INT [32], RREA [24], COTSAE [40], PipEA [38], ZeroEA [17], SelfKG [22], Simple-HHEA [18], AttrGNN [23] and PARIS+ [20]. We were not able to repro-

⁵ <http://csd.uoc.gr/~vefthym/minoanER/datasets/bbcMusic.tar.gz>

⁶ <http://km.aifb.kit.edu/projects/btc-2012/>

⁷ <http://downloads.dbpedia.org/2016-04/>

duce the results of COTSAE [40] from the provided github link [41], so we report the results presented in [40] for the shared datasets. For PipEA, the provided source code [7] only contains a non-iterative (i.e., basic) version.

Traditionally, entity matching has been evaluated with classification metrics (precision, recall) against ground-truth matches [4,6], whereas KG embedding-based EA methods adopted rank-based metrics (e.g., hits at k), likely as a result of the one-to-one assumption. We agree with the criticism of [20] that the total one-to-one assumption is unrealistic, and we note that it makes the problem substantially easier, since the guaranteed existence of a counterpart for every entity is already a strong prior that a real-world matching system does not have. Nonetheless, for evaluating HybEA on an equal basis with existing methods, and in order to re-use the available open-source code as much as possible, we adopt the latter evaluation metrics.

Our experimental evaluation relies on standard effectiveness measures i.e., hits at k ($H@k$) and Mean Reciprocal Rank (MRR). To enable comparison with PARIS+ [20] that reports only precision, recall and F1-score, in Table 1 we consider the F1-score equal to $H@1$ since in the test phase, each source entity gets a list of candidates [31]. Furthermore, we evaluate the effectiveness of the reciprocity filter alone used for enriching the training set. As this filter returns matching decisions, and not rankings of pairs, its performance is assessed via standard classification measures, i.e., precision, recall, and F1-score, cumulatively, across all cycles, per dataset and per component (structural and factual). For example, a component returning 10 correct matches in one cycle and 5 in the next has cumulative precision 100% and recall 15% over 100 test pairs.

3.1 HybEA Performance

In this section, we present and analyze our experimental findings. First, we present the overall effectiveness results in mono- and multilingual datasets, and then proceed with a detailed ablation study. HybEA-R and HybEA-K refer to the variants using RREA and Knowformer as structural backbones, respectively.

Effectiveness Results We can observe the same trend when HybEA is compared to the structural models used as backbones. HybEA-R improves RREA, with relative $H@1$ gains ranging from 11.35% to 96.75%. HybEA-K also improves Knowformer, with gains from 14.8% to 92.77%. The improvement is larger for Knowformer, which means that this model benefits more from the hybrid framework. However, HybEA-K still stays below HybEA-R, suggesting that RREA remains the stronger backbone in this setting.

Results analysis As shown in Appendix B, most datasets in Entity Alignments fall into 3 clusters: monolingual datasets (low structural, high semantic heterogeneity, with BBC-DB as an outlier), multilingual datasets (no particular heterogeneity on either axis), and datasets relaxing the total one-to-one coverage assumption (strong structural but weak semantic heterogeneity) We thus expect structure-based approaches (Knowformer, RREA, PipEA, MTransE, PARIS+)

Table 1: Effectiveness results, with highest score per dataset and measure in boldface, runner-up underlined, best baseline (wrt. H@1) in red. The first iteration of semi-supervised methods reported separately, in italics. PARIS+: We report F1-score as H@1. The last row (Δ) reports the difference, in absolute number and in percentage, of the highest H@1 score (in bold) from the best baseline H@1 score (in red).

Method	Metric	D-W (S)	D-W (D)	D-W (SRPRS-N)	D-W (SRPRS-D)	BBC-DB	ICEWS-WIKI	ICEWS-YAGO
MTransE	H@1	0.260	0.262	0.210	0.347	0.249	0.001	0.000
	H@10	0.540	0.574	0.493	0.680	0.502	0.006	0.001
	MRR	0.35	0.36	0.30	0.46	0.33	0.004	0.001
Knowformer	H@1	0.559	0.840	0.388	0.788	0.289	0.016	0.013
	H@10	0.786	0.941	0.656	0.924	0.506	0.075	0.046
	MRR	0.64	0.87	0.47	0.83	0.37	0.03	0.02
BERT-INT	H@1	0.440	0.426	0.519	0.642	0.925	0.561	0.756
	H@10	0.489	0.485	0.534	0.650	0.937	0.700	0.859
	MRR	0.45	0.44	0.52	0.64	0.93	0.60	0.79
<i>RREA</i> (basic)	H@1	0.655	0.878	0.446	0.817	0.436	0.050	0.026
	H@10	0.884	0.986	0.754	0.962	0.652	0.227	0.136
	MRR	0.74	0.92	0.55	0.87	0.52	0.11	0.06
RREA	H@1	0.718	0.937	0.503	0.881	0.468	0.050	0.027
	H@10	0.900	0.991	0.768	0.977	0.651	0.230	0.142
	MRR	0.79	0.96	0.59	0.91	0.54	0.11	0.06
COTSAE	H@1	-	-	0.709	0.922	-	-	-
	H@10	-	-	0.904	0.983	-	-	-
	MRR	-	-	0.77	0.94	-	-	-
<i>PipEA</i> (basic)	H@1	0.402	0.736	0.241	0.708	0.140	N/A	N/A
	H@10	0.544	0.804	0.371	0.823	0.493	N/A	N/A
	MRR	0.45	0.76	0.29	0.75	0.02	N/A	N/A
ZeroEA	H@1	0.466	0.538	0.469	0.604	0.969	N/A	N/A
	H@10	0.504	0.567	0.488	0.620	0.976	N/A	N/A
	MRR	0.43	0.48	0.45	0.55	0.80	N/A	N/A
SelfKG	H@1	0.542	0.620	0.586	0.734	0.961	0.839	0.806
	H@10	0.707	0.749	0.750	0.852	0.989	0.931	0.867
	MRR	0.59	0.66	0.63	0.77	0.97	0.871	0.828
Simple-HHEA	H@1	0.071	0.077	0.104	0.144	0.098	0.720	0.847
	H@10	0.235	0.327	0.305	0.402	0.291	0.872	0.915
	MRR	0.12	0.15	0.17	0.22	0.16	0.754	0.870
AttrGNN	H@1	0.522	0.570	0.366	0.193	0.311	0.047	0.015
	H@10	0.692	0.657	0.588	0.343	0.539	-	-
	MRR	0.58	0.60	0.44	0.24	0.39	0.09	0.04
PARIS+	H@1	0.841	0.938	0.442	0.834	0.387	0.672	0.687
HybEA-R	H@1	0.989	1.000	0.972	0.997	0.993	0.994	0.994
	H@10	0.997	1.000	0.992	1.000	1.000	0.997	0.997
	MRR	0.99	1.00	0.98	1.00	1.00	1.00	1.00
HybEA-K	H@1	0.920	0.988	0.908	0.968	0.996	0.916	0.941
	H@10	0.969	0.997	0.954	0.987	0.997	0.938	0.944
	MRR	0.93	0.99	0.92	0.97	0.99	0.92	0.94
Δ	H@1	+0.148 (+17.6%)	+0.062 (+6.6%)	+0.263 (+40.6%)	+0.075 (+7.7%)	+0.027 (+3.6%)	+0.155 (+18.5%)	+0.147 (+17.4%)

Table 2: Effectiveness results in multilingual datasets, with highest score per dataset and measure in boldface; runner-up underlined. OOM: out of memory.

Method	FR-EN			JA-EN			ZH-EN		
	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR
MTransE	0.244	0.556	0.335	0.279	0.575	0.349	0.308	0.614	0.364
Knowformer	0.774	0.932	0.832	0.731	0.902	0.793	0.765	0.888	0.811
BERT-INT	0.992	0.998	0.995	0.964	0.991	0.975	0.968	0.990	0.977
RREA (basic)	0.739	0.946	0.816	0.713	0.933	0.793	0.715	0.929	0.794
RREA	0.827	0.966	0.881	0.802	0.952	0.858	0.801	0.948	0.857
ZeroEA	<u>0.998</u>	<u>0.999</u>	<u>0.998</u>	<u>0.982</u>	<u>0.995</u>	<u>0.989</u>	<u>0.985</u>	0.993	<u>0.991</u>
SelfKG	0.957	0.992	0.971	0.813	0.906	0.844	0.742	0.861	0.782
AttrGNN	0.942	0.986	0.959	0.783	0.920	0.834	0.796	0.929	0.845
PARIS+	0.882	-	-	0.824	-	-	OOM	-	-
HybEA-R	0.999	1.000	0.999	0.993	0.999	0.995	0.994	0.999	0.996

to perform well on monolingual datasets and poorly on highly structurally heterogeneous ones (ICEWS-WIKI, ICEWS-YAGO), language-model approaches (BERT-INT, ZeroEA, SelfKG) to favor semantically homogeneous datasets, and most methods to perform correctly on multilingual datasets.

The results in Table 1 confirm this intuition: HybEA-R obtains the best results on all datasets despite their different forms of heterogeneity. On D-W (S)/D-W (D) and D-W (SRPRS-N)/D-W (SRPRS-D) it consistently sur-

Table 3: Reciprocity filter results.

Dataset	struct. first	Pr	Re	Max	F1	fact. first	Pr	Re	Max	F1
D-W (S)	structural	1.0	0.63		0.77	fact.	1.0	0.54		0.70
	factual	1.0	0.23	(0.37)	0.37	struct.	1.0	0.36	(0.46)	0.53
	cumulative	1.0	0.86		0.92	cumulative	1.0	0.90		0.95
D-W (D)	structural	1.0	0.68		0.81	fact.	1.0	0.60		0.75
	factual	1.0	0.22	(0.32)	0.36	struct.	1.0	0.38	(0.40)	0.55
	cumulative	1.0	0.91		0.95	cumulative	1.0	0.98		0.99
D-W (SRPRS-N)	structural	1.0	0.48		0.65	fact.	1.0	0.52		0.69
	factual	1.0	0.33	(0.52)	0.50	struct.	1.0	0.33	(0.48)	0.50
	cumulative	1.0	0.82		0.90	cumulative	1.0	0.86		0.92
D-W (SRPRS-D)	structural	1.0	0.71		0.83	fact.	1.0	0.44		0.61
	factual	1.0	0.13	(0.29)	0.23	struct.	1.0	0.51	(0.57)	0.67
	cumulative	1.0	0.85		0.91	cumulative	1.0	0.95		0.97
BBC-DB	structural	1.0	0.44		0.62	fact.	1.0	0.73		0.85
	factual	1.0	0.49	(0.56)	0.65	struct.	1.0	0.22	(0.27)	0.36
	cumulative	1.0	0.94		0.96	cumulative	1.0	0.95		0.98
ICEWS-WIKI	structural	1.0	0.02		0.04	fact.	1.0	0.92		0.96
	factual	0.99	0.89	(0.98)	0.94	struct.	1.0	0.01	(0.08)	0.02
	cumulative	0.99	0.91		0.95	cumulative	1.0	0.92		0.96
ICEWS-YAGO	structural	1.0	0.01		0.03	fact.	1.0	0.94		0.97
	factual	1.0	0.92	(0.99)	0.96	struct.	1.0	0.003	(0.06)	0.02
	cumulative	1.0	0.94		0.97	cumulative	1.0	0.94		0.97

Cumulative Precision (Pr), Recall (Re), and F1-score (F1) results of the reciprocity filter per dataset and per component, over all iterations for alternative orders of components execution. Max denotes the maximum recall of the second component, i.e., $1 - \text{Re}$ of the first component.

passes the strongest baselines (RREA and COTSAE, respectively), with up to a 40% relative H@1 gain on D-W (SRPRS-N). The gap is most striking on ICEWS-WIKI/ICEWS-YAGO, where extreme structural heterogeneity makes structure-based methods almost ineffective (RREA at 0.050/0.027 H@1) while HybEA-R reaches 0.994 on both, improving the best baseline by more than 17%. BBC-DB is the opposite case: its near-star topology weakens the structural signal and favors language-model methods (ZeroEA at 0.969 H@1), yet HybEA-R still stays at ≥ 0.99 H@1. This confirms that HybEA-R relies on the most informative signal available in each dataset rather than committing to one a priori. The same tendency holds for the multilingual datasets in Table 2: although the baselines are already strong, HybEA-R still obtains the best results on all three, with smaller but consistent improvements.

Reciprocity filter. By comparing the recall obtained by the reciprocity filter with the final H@1 scores, we notice that the vast majority of correct matches identified by HybEA come from the reciprocity filter. Only a very small number of pairs is left to be decided by the bipartite matching applied on the similarity matrix of the last component (Tables 4 and 3). This also explains the very high reliability of the filter, which reaches perfect precision and high recall, with most pairs being added already in the first cycle (Appendix C.2). Results for HybEA-K are reported in Appendix C.3.

It is also worth noting that the component executed second can only work on the pairs that were not matched by the first one. As a consequence, its maximum possible recall is limited by the remaining unmatched part, as shown in the *max* columns of Table 3. In practice, running the factual component first provides a more balanced contribution between the two components and leads to a higher

Table 4: Ablation study for HybEA-R.

Method	Metric	D-W (S)	D-W (D)	D-W (SRPRS-N)	D-W (SRPRS-D)	BBC-DB	ICEWS-WIKI	ICEWS-YAGO
HybEA-R	H@1	0.989	1.000	0.972	0.997	0.993	0.994	0.994
	H@10	0.997	1.000	0.992	1.000	1.000	0.997	0.997
	MRR	0.99	1.00	0.98	1.00	1.00	1.00	1.00
HybEA-R (basic)	H@1	0.986	0.999	0.963	0.995	0.991	0.994	0.994
	H@10	0.997	1.000	0.989	1.000	1.000	0.997	0.997
	MRR	0.99	1.00	0.97	1.00	0.99	1.00	1.00
HybEA-R (basic; str. first)	H@1	0.779	0.866	0.742	0.797	0.832	0.985	0.953
	H@10	0.802	0.883	0.781	0.809	0.843	0.995	0.976
	MRR	0.78	0.87	0.75	0.80	0.83	0.98	0.96
HybEA-R (struct. first)	H@1	0.872	0.915	0.829	0.853	0.942	0.985	0.953
	H@10	0.881	0.922	0.847	0.860	0.944	0.995	0.976
	MRR	0.87	0.91	0.83	0.85	0.94	0.98	0.96
HybEA-R (w/o fact.)	H@1	0.655	0.878	0.446	0.817	0.436	0.050	0.026
	H@10	0.884	0.986	0.754	0.962	0.652	0.227	0.136
	MRR	0.74	0.92	0.55	0.87	0.52	0.11	0.06
HybEA-R (w/o struct.)	H@1	0.558	0.621	0.513	0.444	0.748	0.926	0.948
	H@10	0.612	0.672	0.592	0.488	0.769	0.974	0.975
	MRR	0.79	0.82	0.70	0.70	0.91	0.94	0.96

cumulative F1 on all datasets. On D-W (S), for example, the structural-first setting reaches $H@1 = 0.87$, with a filter recall of 0.86, while the factual-first setting reaches $H@1 = 0.99$, with a filter recall of 0.90. The same tendency is observed from D-W (D) to BBC-DB, with gains between 2% and 11%, and factual-first remains the best setting even on ICEWS-WIKI and ICEWS-YAGO.

The two components are complementary and adapt to the heterogeneity of the datasets. The structural component brings most of the evidence on dense datasets, reaching 0.51 recall on D-W (SRPRS-D), while the factual component is more effective when structural heterogeneity increases, with 0.54 on D-W (S), 0.60 on D-W (D), and 0.73 on BBC-DB. In the extreme case of ICEWS-WIKI/ICEWS-YAGO, only the factual component contributes. This confirms that HybEA-R does not prioritize one source of information a priori, but rather exploits the most reliable one depending on the dataset.

Ablation Study. Finally, the ablation study confirms the contribution of each component of HybEA-R (Table 4); the study for HybEA-K, with even larger drops, is reported in the Appendix (Table 12). Removing the factual component reduces HybEA-R to RREA with the reciprocity filter, yielding the same results as RREA (basic), and decreases $H@1$ by 12.2% to 96.8% on average. Removing the structural component makes the model rely only on factual evidence and is generally worse than using the structural model alone, except on ICEWS-WIKI/ICEWS-YAGO. This confirms that the factual-first strategy benefits from the interaction between the two models. Factual-first also outperforms structural-first on all datasets, which justifies its use as the default ordering. The single-cycle basic version remains below the full pipeline; the difference is small for HybEA-R because the basic version is already near-perfect, but it becomes clearer for the structural-first variants and even larger for HybEA-K.

Efficiency. A complete efficiency analysis is provided in Appendix C.4, reporting for each dataset and each component the mean time per epoch, the total number of epochs, the number of semi-supervision cycles and the resulting total time (Tables 9 and 10), as well as a comparison of the training cost of HybEA against the baselines (Table 14).

4 Conclusion

In this paper, we have introduced HybEA, an open-source, hybrid EA framework that can be build on top of any pure structural alignment method. The combination of both factual and structural components allows us to cope with high-heterogeneous datasets, with one model capturing structural and one model capturing factual evidence of alignments. We experimentally demonstrated that HybEA systematically outperforms all state-of-the-art baseline methods, including BERT-INT, Knowformer and RREA, by as far as 29% Hits@1, obtaining almost perfect scores on every dataset.

Our choice of two separate models for tackling semantic and structural heterogeneity of KGs can also be considered as an important step towards the need to explain matching decisions [35,34,26] at different granularity levels to debug, refine, and validate the alignment process: (i) *coarse-grained*: it makes it easy to see the provenance of the returned matches by the structural vs the factual component. (ii) *fine-grained*: the factual model could highlight the most influential attributes in finding a match (e.g., birthDate, foaf:name etc), while the structural model could report the most influential relations that support the matches (e.g., both are connected to similar subgraphs).

Assessing the impact of structural diversity in EA is only a first step in tackling matching bias of KGs. There is strong evidence that entities with rich and high-quality factual information are more likely to be correctly matched, opening a new research work related to group fairness for EA. We plan to extend our graph sampling method [14] to also consider the diversity of factual information (i.e., literal values) between two KGs and assess the robustness of EA methods.

References

1. Ahmetaj, S., Efthymiou, V., Fagin, R., Kolaitis, P.G., Lei, C., Özcan, F., Popa, L.: Ontology-enriched query answering on relational databases. In: AAAI (2021)
2. Chen, M., Tian, Y., Chang, K., Skiena, S., Zaniolo, C.: Co-training embeddings of knowledge graphs and entity descriptions for cross-lingual entity alignment. In: IJCAI (2018)
3. Chen, M., Tian, Y., Yang, M., Zaniolo, C.: Multilingual knowledge graph embeddings for cross-lingual knowledge alignment. In: IJCAI (2017)
4. Christen, P.: Data Matching - Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Data-Centric Systems and Applications (2012)
5. Christophides, V., Efthymiou, V., Palpanas, T., Papadakis, G., Stefanidis, K.: An overview of end-to-end entity resolution for big data. ACM Comput. Surv. (2021)
6. Christophides, V., Efthymiou, V., Stefanidis, K.: Entity Resolution in the Web of Data (2015)
7. wyy code: Pipea (2024), <https://github.com/wyy-code/PipeEA>
8. Demsar, J.: Statistical comparisons of classifiers over multiple data sets. J. Mach. Learn. Res. **7** (2006)
9. Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: NAACL-HLT (2019)

10. Dong, X., Gabrilovich, E., Heitz, G., Horn, W., Lao, N., Murphy, K., Strohmman, T., Sun, S., Zhang, W.: Knowledge vault: a web-scale approach to probabilistic knowledge fusion. In: SIGKDD (2014)
11. Efthymiou, V., Papadakis, G., Stefanidis, K., Christophides, V.: MinoanER: Schema-agnostic, non-iterative, massively parallel resolution of web entities. In: EDBT (2019)
12. Efthymiou, V., Stefanidis, K., Christophides, V.: Big data entity resolution: From highly to somehow similar entity descriptions in the web. In: 2015 IEEE International Conference on Big Data (Big Data) (2015)
13. Efthymiou, V., Stefanidis, K., Christophides, V.: Big data entity resolution: From highly to somehow similar entity descriptions in the web. In: IEEE BigData (2015)
14. Fanourakis, N., Efthymiou, V., Christophides, V., Kotzinos, D., Pitoura, E., Stefanidis, K.: Structural bias in knowledge graphs for the entity alignment task. In: ESWC (2023)
15. Fanourakis, N., Efthymiou, V., Kotzinos, D., Christophides, V.: Knowledge graph embedding methods for entity alignment: experimental review. *Data Min. Knowl. Discov.* (2023)
16. Guo, L., Sun, Z., Hu, W.: Learning to exploit long-term relational dependencies in knowledge graphs. In: ICML (2019)
17. Huo, N., Cheng, R., Kao, B., Ning, W., Haldar, N.A.H., Li, X., Li, J., Najafi, M.M., Li, T., Qu, G.: Zeroea: A zero-training entity alignment framework via pre-trained language model. *Proc. VLDB Endow.* (2024)
18. Jiang, X., Xu, C., Shen, Y., Wang, Y., Su, F., Shi, Z., Sun, F., Li, Z., Guo, J., Shen, H.: Toward practical entity alignment method design: Insights from new highly heterogeneous knowledge graph datasets. In: WWW (2024)
19. Lample, G., Conneau, A., Ranzato, M., Denoyer, L., Jégou, H.: Word translation without parallel data. In: ICLR (2018)
20. Leone, M., Huber, S., Arora, A., García-Durán, A., West, R.: A critical re-evaluation of neural methods for entity alignment. *PVLDB* (2022)
21. Li, G., Sun, Z., Hu, W., Cheng, G., Qu, Y.: Position-aware relational transformer for knowledge graph embedding. *IEEE Trans. Neural Networks Learn. Syst.* (2024)
22. Liu, X., Hong, H., Wang, X., Chen, Z., Kharlamov, E., Dong, Y., Tang, J.: Selfkg: Self-supervised entity alignment in knowledge graphs. In: WWW (2022)
23. Liu, Z., Cao, Y., Pan, L., Li, J., Liu, Z., Chua, T.: Exploring and evaluating attributes, values, and structures for entity alignment. *CoRR* (2020)
24. Mao, X., Wang, W., Xu, H., Wu, Y., Lan, M.: Relational reflection entity alignment. In: CIKM (2020)
25. Papadakis, G., Efthymiou, V., Thanos, E., Hassanzadeh, O., Christen, P.: An analysis of one-to-one matching algorithms for entity resolution. *VLDB J.* (2023)
26. Pradhan, R., Lahiri, A., Galhotra, S., Salimi, B.: Explainable AI: foundations, applications, opportunities for data management research. In: SIGMOD (2022)
27. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. In: EMNLP-IJCNLP (2019)
28. Suchanek, F.M., Abiteboul, S., Senellart, P.: PARIS: probabilistic alignment of relations, instances, and schema. *Proc. VLDB Endow.* (2011)
29. Sun, Z., Hu, W., Li, C.: Cross-lingual entity alignment via joint attribute-preserving embedding. *CoRR* (2017)
30. Sun, Z., Hu, W., Zhang, Q., Qu, Y.: Bootstrapping entity alignment with knowledge graph embedding. In: IJCAI (2018)

31. Sun, Z., Zhang, Q., Hu, W., Wang, C., Chen, M., Akrami, F., Li, C.: A benchmarking study of embedding-based entity alignment for knowledge graphs. *Proc. VLDB Endow.* (2020)
32. Tang, X., Zhang, J., Chen, B., Yang, Y., Chen, H., Li, C.: BERT-INT: A bert-based interaction model for knowledge graph alignment. In: *IJCAI*. pp. 3174–3180 (2020)
33. Tarus, J.K., Niu, Z., Mustafa, G.: Knowledge-based recommendation: a review of ontology-based recommender systems for e-learning. *Artif. Intell. Rev.* (2018)
34. Teofili, T., Firmani, D., Koudas, N., Martello, V., Merialdo, P., Srivastava, D.: Effective explanations for entity resolution models. In: *ICDE* (2022)
35. Teofili, T., Firmani, D., Koudas, N., Merialdo, P., Srivastava, D.: CERTEM: explaining and debugging black-box entity resolution systems with CERTA. *Proc. VLDB Endow.* (2022)
36. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *NeurIPS* (2017)
37. Wang, C., Huang, Z., Wan, Y., Wei, J., Zhao, J., Wang, P.: Fualign: Cross-lingual entity alignment via multi-view representation learning of fused knowledge graphs. *Inf. Fusion* (2023)
38. Wang, Y., Tang, W., Sun, H., Zhuang, Z., Fu, X., Wang, J., Qi, Q., Liao, J.: Understanding and guiding weakly supervised entity alignment with potential isomorphism propagation. *CoRR* (2024)
39. Wang, Z., Yang, J., Ye, X.: Knowledge graph alignment with entity-pair embedding. In: *EMNLP* (2020)
40. Yang, K., Liu, S., Zhao, J., Wang, Y., Xie, B.: COTSAE: co-training of structure and attribute embeddings for entity alignment. In: *AAAI* (2020)
41. ykpku: Cotsa (2019), <https://github.com/ykpku/COTSA>
42. Zeng, K., Li, C., Hou, L., Li, J., Feng, L.: A comprehensive survey of entity alignment for knowledge graphs (2021)
43. Zeng, W., Zhao, X., Tan, Z., Tang, J., Cheng, X.: Matching knowledge graphs in entity embedding spaces: An experimental study. *IEEE Transactions on Knowledge and Data Engineering* (2023)
44. Zhang, D., Rubinstein, B.I.P., Gemmell, J.: Principled graph matching algorithms for integrating multiple data sources. *CoRR* (2014)
45. Zhang, Q., Sun, Z., Hu, W., Chen, M., Guo, L., Qu, Y.: Multi-view knowledge graph embedding for entity alignment. In: *IJCAI* (2019)
46. Zhang, R., Trisedya, B.D., Li, M., Jiang, Y., Qi, J.: A benchmark and comprehensive survey on knowledge graph entity alignment via representation learning. *VLDBJ.* (2022)

A Related Work

Despite the many names and settings of EA problem, most related works can be broadly classified with respect to their focus on the structural information (e.g., entity relationships through foreign keys, dependency graphs, or relations/edges in KGs) or on the factual information of entities (e.g., their literal values). In the sequel, we will position HybEA in the context of these two general families of EA systems.

A.1 Structure-based approaches.

Knowing that two candidate matches have many matching neighbors, even better, through semantically similar edges, provides strong evidence that those candidates are also matches; especially when the involved edges represent functional relations [28].

In this respect, there are several EA methods that learn vector representations (embeddings) of nodes (entities), encoding structurally similar nodes close in the vector space, while dissimilar far. Like MTransE [3], which encodes the entities of the two KGs in different vector spaces using TransE, and then exploits their local structure before transitioning between them.

Moreover, several Graph Neural Networks (GNNs) have been proposed to account a wider range of each node neighborhood, than translation-based methods. However, this comes with the risk of incorporating noisy or even irrelevant to the task information to the node embeddings, from distant neighbors. In this respect, PipEA [38] modifies GNNs propagation strategy in order to allow the information to also pass from one graph to the other and thus to compute a similarity measure between subgraphs of both KGs. RREA [24] constraints the GNNs transformation matrix to be orthogonal and hence preserve shape similarity. It also proposes a semi-supervised setting that adds to the training set mutually nearest neighbors, while it also adopts an attention mechanism for filtering the information of noisy distant neighbors.

Finally, there also exist a few structural methods that rely neither on translational models, nor on GNNs. Knowformer [21] exploits Transformers to create entity embeddings using individual relation triplets of KGs. To integrate structural information such as direction and local information, Transformers are extended with a position-aware structural component, called relational composition. PARIS+ [20,28] relies on a probabilistic method to align entities, by statistically estimating matching probabilities.

Structured-based methods rely on the rather unrealistic assumption that equivalent entities from different KGs possess similar neighboring KG structures (and in turn similar embeddings). Hence, they leverage the seed entity pairs as anchors and progressively project individual KG embeddings into a unified space through training, resulting in the unified entity representations [43]. Compared to this family of EA methods, HybEA is able to adapt to different degrees of structural and semantic heterogeneity exhibited by mono- and multilingual KGs (see Table 6).

A.2 Joint approaches.

As the number of attributes significantly exceeds that of relations in real KGs (see Table 5), some methods jointly consider the factual and structural information of entities. Although they have been empirically proven to be highly effective [15], their performance significantly drops in KGs exhibiting high heterogeneity at both structural and factual levels.

JAPE [29] combines structure embedding and attribute embedding to match entities in different KGs, using TransE and Skip-Gram, respectively. MultiKE [45] considers three views of KGs, distinguishing the name, relations and attributes of entities. These views are jointly learned using a loss function that weighs all three equally. This method assumes high homogeneity in both attribute names and values, thus we expect MultiKE to perform better in datasets originating from the same or similar data sources (e.g., Wikipedia-based, or synthetic KGs). On the other hand, KDCoE [2] is a semi-supervised method that co-trains two embedding models, one on the structural part of entities (MTransE [3]) and the other on the textual descriptions of the entities. Similarly, COTSAE [40] co-trains a structural model along with a general attention-based factual model and employs a bipartite graph matching algorithm to find the global optimum result. COTSAE’s attention mechanism can capture only short-term dependencies, and thus not all attributes of KGs are exploited for learning the attribute attentions of an entity. In addition, the new matches of the semi-supervised COTSAE rely solely on the refined structured entity embeddings, which are usually false positives (especially when structural heterogeneity is high), introducing noise to the model, resulting to low adaptability of the method in terms of structural and semantic heterogeneity.

A.3 Language Model-based approaches.

Pre-trained language Models (PLM) have been employed to cope with the semantic heterogeneity of KGs. AttrGNN [23] leverages masked language models like BERT [9] to learn attribute value and name embeddings (literal, digital and name channel) and multi-layer GCN for learning the structure embeddings (structure channel). It then ensemble the information provided by the aforementioned channels, as well as an attribute attention mechanism based on GAT.

On the other hand, BERT-INT [32] leverages masked language models to contextualize literal descriptions of entities in embeddings. More precisely, BERT-INT fine-tunes BERT to encode entity names, textual descriptions, and attribute values of entities, performing pairwise neighbor-view and attribute-view interactions to obtain entity matching scores. To reduce the number of unnamed nodes for which the method considers only random embeddings, BERT-INT relies on additional textual information not necessarily included in the original datasets. ZeroEA [17] also leverages BERT over entire entity neighborhoods along with a natural language translator to obtain an embedding that is directly used to compute the similarity scores.

Simple-HHEA rely on random walks [37] to encode the structure of entities and on BERT to initialize the entity embeddings using their name. It also uses an entity time encoder for exploiting the temporal information of KGs (if it is available). Finally, to overcome the scarcity of manually labeled entity pairs, SelfKG [22] adapts the NCE loss for self-supervision with self-negative sampling along a GAT and LaBSE for initializing the entity embeddings with their names. As it is demonstrated later in the experimental results (see Table 1), despite the power of the PLMs, PLM-based methods require low semantic heterogeneity to provide satisfactory effectiveness.

B Dataset and Heterogeneity Analysis

In this section, we provide a detailed analysis of the different datasets we used, under the heterogeneity scope. For this purpose, after an analysis of the different datasets used, we propose different metrics that exhibit the different levels of heterogeneity, both structurally and semantically.

Table 5: Statistics per dataset. $|E|$: number of entities, $|X|$: relation triples, $|Y|$: attribute triples, $|R|$: relations, $|A|$: attributes. $wccR$ and $maxCS$ are defined in [14] as the ratio of number of weakly connected components over $|E|$, and the size of the biggest weakly connected component over $|E|$, respectively.

Dataset	$ E $	$ X $	$ Y $	$ R $	$ A $	$maxCS$	$wccR$	nodes w/o name	train/val/test
D-W (S)									3,000/1,500/10,500
KG_1	15,000	38,265	68,258	248	342	0.95	0.01	0	
KG_2	15,000	42,746	138,246	169	649	0.93	0.02	3,707	
D-W (D)									3,000/1,500/10,500
KG_1	15,000	73,983	66,813	167	175	0.99	0.0006	0	
KG_2	15,000	83,365	175,686	121	457	0.99	0.001	2,955	
D-W (SRPRS-N)									3,000/1,500/10,500
KG_1	15,000	38,421	71,957	253	363	0.98	0.005	0	
KG_2	15,000	40,159	136,315	144	652	0.99	0.003	6,581	
D-W (SRPRS-D)									3,000/1,500/10,500
KG_1	15,000	68,598	62,636	220	256	0.99	0.002	0	
KG_2	15,000	75,465	184,332	135	531	0.99	0.002	5,096	
BBC-DB									1,879/939/6,578
KG_1	9,395	15,478	18,165	9	4	0.31	0.18	46	
KG_2	9,395	45,561	149,720	98	723	0.78	0.07	0	
FR-EN									3,000/1,500/10,500
KG_1	19,661	105,998	528,665	903	4,547	0.99	0.0007	0	
KG_2	19,993	115,722	576,543	1,208	6,422	0.99	0.0008	0	
JA-EN									3,000/1,500/10,500
KG_1	19,814	77,214	354,619	1,299	5,882	0.98	0.001	9,429	
KG_2	19,780	93,484	497,230	1,153	6,066	0.99	0.0008	0	
ZH-EN									3,000/1,500/10,500
KG_1	19,388	70,414	379,684	1,701	8,113	0.98	0.002	17,280	
KG_2	19,572	95,142	567,755	1,323	7,173	0.99	0.0009	0	
ICEWS-WIKI									1,012/506/3,540
KG_1	11,047	3,527,881	11,047	272	1	0.99	0.0002	0	
KG_2	15,831	198,257	15,831	226	1	1.0	0.00006	0	
ICEWS-YAGO									5,142/506/13,176
KG_1	26,863	4,192,555	26,863	272	1	0.99	0.0002	0	
KG_2	22,555	107,118	22,555	41	1	0.99	0.001	0	

B.1 Datasets features

In this section, we describe datasets main features, as shown in Table 5.

Table 5 details the following characteristics of datasets:

- Number of entities $|E|$: the number of nodes in each KG, each describing a real-world entity.
- Number of relation triples $|X|$: the number of triples whose head and tail (subject and object) are both entities.
- Number of attribute triples $|Y|$: the number of triples whose head (subject) is an entity and tail (object) is a literal value.
- Number of relations $|R|$: the number of distinct relations appearing in a KG, i.e., the number of relation types.
- Number of attributes $|A|$: the number of distinct attributes appearing in a KG, i.e., the number of attribute types.
- Largest weakly connected component $maxCS$: defined in [14] as $\max_{CC \in wcc}(|CC|)/|E|$, where wcc is the set of weakly connected components in a KG. Intuitively, the higher the size of the largest component (measured in number of nodes in that component), the easier it is to perform EA.
- Weakly connected components ratio $wccR$: defined in [14] as $|wcc|/|E|$, where $|wcc|$ is the number of weakly connected components in a KG. Intuitively, for a fixed $|E|$, the higher the number of connected components, the more difficult it is to perform EA.
- Nodes without a name: the number of nodes for which no attribute can be used as an entity label. The set of attributes that can be used for that purpose (e.g., `rdfs:label`, `skos:name`, `foaf:name`) may be selected manually per KG, or detected automatically by statistical methods [11].
- Data splitting in train/validation/test: the number of positively labeled (i.e., matching) seed alignment pairs used for the training, validation, and test set, respectively.

B.2 Heterogeneity metrics

In this section, we propose different metrics that could enrich our understanding of the levels of heterogeneity the datasets exhibit. To this order, we propose 4 metrics: 2 structural metrics and 2 semantic metrics. Each of them is computed by only taking into account the aligned pairs, with the idea that structurally and/or semantically similar entities are more likely to be matched than dissimilar ones. Thus, the difficulties come when the entities that have to be matched are structurally and/or semantically dissimilar. Table 6 reports for each dataset the following metrics:

- **Jaccard index between the matched neighbors (J)**: Intuitively, matching entities whose neighbors are also respectively matched is easier than matching entities for which none of their neighbors are respectively matched. We thus define the set of entities in the neighborhood of u that are matched with an entity in the neighborhood in v as $M_{u,v} = \{(e_i, e_j) \in E_1 \times E_2 \mid$

Table 6: Heterogeneity metric per dataset. See Section B.2 for information about the different computed metrics.

Dataset	Jaccard	LDMA	D Mean degree	KG_1 Mean degree	KG_2 Levenshtein on attributes	Levenshtein on names
D-W (S)	0.63	2.08	4.21	5.14	0.58	0.50
D-W (D)	0.67	3.22	8.14	9.87	0.64	0.55
D-W (SRPRS-N)	0.62	1.51	4.38	4.27	0.72	0.50
D-W (SRPRS-D)	0.68	2.13	7.29	7.81	0.73	0.59
BBC-DB	0.67	1.33	1.79	2.93	0.72	0.71
FR-EN	0.41	4.11	10.63	11.24	0.72	0.82
JA-EN	0.41	3.28	7.51	8.95	0.63	0.17
ZH-EN	0.41	3.67	7.04	9.20	0.67	0.02
ICEWS-WIKI	0.02	90.21	102.19	22.45	0.85	0.92
ICEWS-YAGO	0.03	43.77	47.76	5.94	0.41	0.95

$e_i \equiv e_j, e_i \in \mathcal{N}(u), e_j \in \mathcal{N}(v)$ where u and v are two aligned entities and $\mathcal{N}(u)$ and $\mathcal{N}(v)$ their respective neighborhood. Our Jaccard index can then be defined as

$$J = \frac{1}{|M|} \sum_{(u,v) \in M} \frac{|M_{u,v}|}{|\mathcal{N}(u)| + |\mathcal{N}(v)| - |M_{u,v}|},$$

where M is the set of matching pairs.

- **Local Degree Mean Absolute Deviation (LDMA)**: The Jaccard index might be too restrictive in some cases, e.g., outside the one-to-one constraint, where matching entities might be far from each other in the same graph, leading to a poor Jaccard index. We thus compare the degree difference between two matched entities:

$$\text{LDMA} = \frac{1}{|M|} \sum_{(u,v) \in M} ||\mathcal{N}(u)| - |\mathcal{N}(v)||.$$

In order to have a more precise idea on the impact of this metric, we also compute the mean degree of each graph.

- **Levenshtein Similarity of Attributes and Names (lev)**: Concerning the semantic similarity, we computed the Levenshtein similarity independently on both attribute values a and entity names b :

$$\text{lev}_{\text{index}}(a, b) = \frac{|a| + |b| - \text{lev}_{\text{distance}}(a, b)}{|a| + |b|}.$$

The name similarity is pretty straightforward for evaluating how semantically similar matched entities are. However, it might not be sufficient enough, e.g. in the case of multi-lingual datasets, where the names will be necessarily different. We thus also compute the Levenshtein similarity between the attributes of matching entities, first because their diversity could allow to bypass the language barrier and alphabet differences, e.g., with dates, and second because they are also one of the main focus of our semantic component.

These metrics are descriptive rather than validated measures of EA difficulty. Character-level Levenshtein similarity is sensitive to writing-system differences, so the JA-EN and ZH-EN name similarities should be interpreted cautiously. For reproducibility, we summarize Table 6 using the following heuristic categories: structural heterogeneity is Low for $J \geq 0.6$, Medium for $0.3 \leq J < 0.6$, and High otherwise. Semantic heterogeneity is based on the name similarity when the writing systems are comparable, and on the attribute similarity otherwise: Low above 0.70, Medium between 0.60 and 0.70, and High below 0.60. These thresholds are descriptive, neither learned nor used by HybEA.

B.3 Datasets Analysis

In terms of structural heterogeneity, we observe three clusters according to the different metrics we computed, in particular our Jaccard index. With this metric, the monolingual datasets (D-W (S), D-W (D), D-W (SRPRS-N), D-W (SRPRS-D) and BBC-DB) stand out with a low structural heterogeneity, as shown by a Jaccard index between 0.63 and 0.68, which means that 2/3 of each aligned entity’s neighbors are also matched. In terms of Local Degree Mean Absolute Deviation in the mono-lingual cluster, we first can see the impact of the Dense datasets (D-W (D) and D-W (SRPRS-D)) compared to the Sparse datasets (D-W (S) and D-W (SRPRS-N)). In both cases, despite increasing largely the mean degree of both graphs, the Local Degree Deviation does not increase in the same proportion, which might indicate a lower structural heterogeneity for the Dense datasets. Concerning BBC-DB, the low value of LDMAD has to be compared with the low value of mean degree in both graphs, and in particular KG_1 . This makes BBC-DB very particular, with very few neighbors per entity, which is supported by the low number of relation triples in this dataset, as it can be seen in Table 5. In conclusion, the mono-lingual cluster can be subdivided into 3 subsets, with the Dense datasets being the least structurally heterogeneous, followed by the Sparse datasets and finally BBC-DB in its own subset.

In terms of semantic heterogeneity, BBC-DB stands out once again among the monolingual datasets by exhibiting the highest Levenshtein similarity, in particular on the names, showing a particularly low semantic heterogeneity on this dataset. It must also be compared with the number of node without names, almost none in the case of BBC-DB, while significant for the other monolingual datasets. In conclusion, among the monolingual datasets, BBC-DB is really particular, showing a very low mean degree on both graphs, which might bias its structural heterogeneity, while also showing a low semantic heterogeneity.

The multilingual datasets are then forming the second cluster, with a medium structural heterogeneity and a Jaccard index equal to 0.41 for all of them. FR-EN has a high deviation in number of nodes but also a high degree on both graphs on average, which once taken into account show a lower structural heterogeneity compared to the other multilingual datasets. On the semantic heterogeneity, the three of them are really different on the name level, with FR-EN having among the lowest semantic heterogeneity while JA-EN and ZH-EN exhibit the highest semantic heterogeneity among all datasets. This can first be explained by the

alphabet difference, which is not taken into account by the Levenshtein distance, and secondly by the high number of unnamed entities in both JA-EN and ZH-EN. Multilingual datasets finally exhibit an average Levenshtein similarity of attribute values.

The last cluster is formed by the two datasets, ICEWS-WIKI and ICEWS-YAGO, which relax the total one-to-one assumption through partial overlap and exhibit the most important structural heterogeneity. This can be explained by the low overlap ratio between each dataset and the high density of each graph, as shown by the mean degree of both graphs, but this also means that the aligned entities are not localized in one section of each graph, but more likely distilled in the whole graph. However, those graphs present the lowest semantic heterogeneity on the name level, which seems to indicate that the aligned entities share very similar names. The two datasets diverge for the Levenshtein similarity on the attribute level, which seems to indicate ICEWS-YAGO has a lower global semantic heterogeneity than ICEWS-WIKI, and thus might be considered a more challenging dataset.

It might however be important to notice that, whether the datasets exhibit a strong structural or semantic heterogeneity, being able to consider both is mandatory. First in our analysis, no dataset appeared as purely structural or purely semantic (even though ICEWS-WIKI and ICEWS-YAGO could arguably be close to such definition), it is thus necessary to consider them both. Second, our analysis does not consider the variability with a graph. Indeed, some graphs may for example exhibit a low semantic heterogeneity in general, but still have some local parts that are highly semantically heterogeneous. For those two reasons, it is thus important to consider both heterogeneities.

To conclude this section, we observe a high diversity within the characteristics of each dataset. Apart from BBC-DB, the monolingual datasets seem to exhibit a lower structural heterogeneity and a higher semantic heterogeneity than the other datasets, which could indicate that structure-based approaches might perform better on those datasets. On the other side, the very high structural heterogeneity of ICEWS-WIKI and ICEWS-YAGO with a very low semantic heterogeneity indicate that methods that do not rely exclusively on the structure should perform better than structure-based approaches. Finally, multilingual datasets provide neither high structural heterogeneity nor high semantic heterogeneity, which might explain their popularity in the experimental evaluation of embedding-based EA methods. Table 7 summarizes the different levels of heterogeneity per dataset.

C Additional Experimental Results

In this section, it is first presented the effectiveness ranking of EA methods. Then we provide a robustness analysis of HybEA-R and HybEA-K, assessing the robustness of our semi-supervised method in newly generated matching pairs. In addition, for completeness purposes, we extend our reciprocity filter analysis of Section 3.1 including results from HybEA-K, with also an efficiency analysis of

Table 7: Summary of the different types and levels of heterogeneity per dataset.

Dataset	Semantic heterogen.	Structural heterogen.
D-W (S)	High	Low
D-W (D)	High	Low
D-W (SRPRS-N)	High	Low
D-W (SRPRS-D)	High	Low
BBC-DB	Low	Low
FR-EN	Low	Medium
JA-EN	Medium	Medium
ZH-EN	Medium	Medium
ICEWS-WIKI	Low	High
ICEWS-YAGO	Low	High

both HybEA-R and HybEA-K and an ablation study showing the impact of each HybEA-K component in its effectiveness improvement. Last but not least, we present an ablation study on different initialization techniques of HybEA-R.

C.1 Effectiveness ranking of EA methods

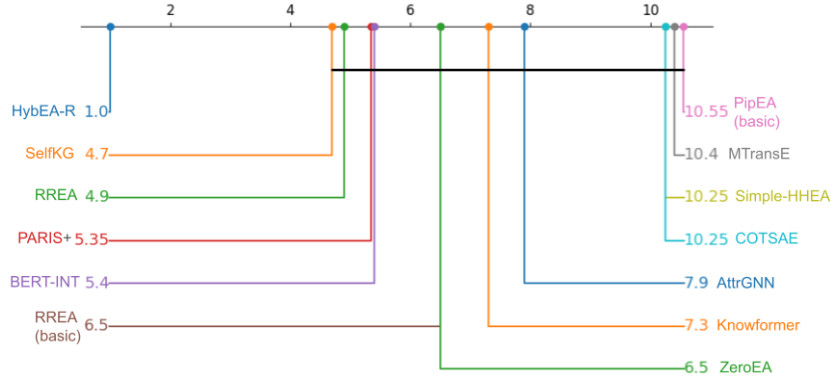


Fig. 4: Statistical significance rankings of EA methods according to H@1.

To infer a statistically significant ranking of methods, we rely on the non-parametric Friedman test [8]. With a p-value of 2.84×10^{-9} for Hits@1, we reject the null hypothesis H_0 at the 5% confidence level. We then apply the Bonferroni post-hoc test to compare methods pairwise. A difference is significant if their average ranks differ by more than the critical distance (CD), computed as 3.93 for 13 methods over 10 datasets at $\alpha = 0.05$.

Figure 4 shows that HybEA-R outperforms all other methods, with rank differences exceeding the CD. SelfKG and RREA are the closest, but still below the threshold. MTransE and PipEA(basic) rank the lowest.

C.2 Robustness of semi-supervision

In Figure 5, we demonstrate H@1 of HybEA-K (Figure 5a) and HybEA-R (Figure 5b) in each semi-supervision cycle, assessing the robustness of our semi-supervised method to the newly generated matching pairs of each cycle. As we can see from the figure, HybEA-K is improved through the cycles in all datasets. The largest improvement is observed from the first to the second cycle, while in the next cycles, the improvement is substantially smaller, due to the rapidly decreasing number of newly generated matching pairs. H@1 is not dropping through cycles, which means that the newly generated matches do not introduce noise. In Figure 5b we demonstrate H@1 of HybEA-R, in each semi-supervision cycle, assessing the robustness of our semi-supervised method to the newly generated matching pairs of each cycle. As we can see from the figure, HybEA-R is improved through the cycles in mainly more datasets. However, we observe a drop in certain cycles for HybEA-R, which could be attributed to the fact that the initial performance is already very high and leave only small space for improvement. Although certain cycles show slight drops, HybEA-R continues to improve from the first to the last cycle, indicating that the method remains effective throughout the process. In both figures, ICEWS-WIKI and ICEWS-YAGO are two points, since they only run in one semi-supervision cycle.

C.3 Reciprocity filter analysis for HybEA-K

In Table 8, we extend the reciprocity filter analysis of Section 3.1, on HybEA-K. More precisely, as described in the corresponding section, it is essential to note that the order of the components is determining the results of the table, since the second component is only called for the pairs that the first component could not match with the reciprocity filter. In other words, the highest achievable recall by the second component is 1 minus the recall of the first component, expressed by the (max) column.

As in Table 3, the left half of the table shows the results obtained by running the structural component first and the right half of the table shows the results obtained by running the factual component first (i.e., the default setting). In both cases, we also report the maximum achievable recall (max) by the second component. By comparing the cumulative F1 scores for the same dataset in the two variations, we notice that by executing the structural component first, we achieve better results than executing the factual component first, in all datasets. We also observe that the contribution of the factual component, when executed second, is very limited, since the effectiveness of the structural component does not allow much room for improvement. On the other hand, when executing the factual component first, the contribution of the two components is more balanced. Note that those results only reflect the effectiveness of the reciprocity filter and they do not reflect the overall evaluation of HybEA-K, although the reciprocity filter largely impacts the overall effectiveness of HybEA-K.

Table 8: Cumulative Precision (Pr), Recall (Re), and F1-score (F1) results of the reciprocity filter per dataset and per component, over all iterations, for HybEA-K. Separate evaluation for structural component ran first and for factual component ran first. Maximum recall (max) of the second component is 1 - Re of the first component.

Dataset	structural first	Pr	Re (max)	F1	factual first	Pr	Re (max)	F1
D-W (S)	structural	1.0	0.75	0.86	fact.	1.0	0.54	0.70
	factual	1.0	0.10 (0.15)	0.18	struct.	1.0	0.28 (0.46)	0.44
	cumulative	1.0	0.85	0.92	cumulative	1.0	0.82	0.90
D-W (D)	structural	1.0	0.94	0.97	fact.	1.0	0.60	0.75
	factual	1.0	0.03 (0.06)	0.06	struct.	1.0	0.35 (0.40)	0.51
	cumulative	1.0	0.97	0.99	cumulative	1.0	0.95	0.98
D-W (SRPRS-N)	structural	1.0	0.60	0.75	fact.	1.0	0.52	0.68
	factual	1.0	0.26 (0.40)	0.41	struct.	1.0	0.31 (0.48)	0.48
	cumulative	1.0	0.86	0.93	cumulative	1.0	0.83	0.91
D-W (SRPRS-D)	structural	1.0	0.88	0.94	fact.	1.0	0.44	0.61
	factual	1.0	0.05 (0.12)	0.10	struct.	1.0	0.48 (0.56)	0.65
	cumulative	1.0	0.93	0.96	cumulative	1.0	0.92	0.96
BBC-DB	structural	1.0	0.97	0.98	fact.	1.0	0.73	0.85
	factual	1.0	0.01 (0.03)	0.02	struct.	1.0	0.22 (0.27)	0.36
	cumulative	1.0	0.98	0.99	cumulative	1.0	0.95	0.97
ICEWS-WIKI	structural	1.0	0.05	0.11	fact.	0.99	0.91	0.95
	factual	0.99	0.85 (0.95)	0.92	struct.	1.0	0.002 (0.09)	0.005
	cumulative	0.99	0.91	0.95	cumulative	0.99	0.91	0.95
ICEWS-YAGO	structural	1.0	0.01	0.02	fact.	1.0	0.93	0.96
	factual	1.0	0.92 (0.99)	0.96	struct.	1.0	0.002 (0.07)	0.002
	cumulative	1.0	0.94	0.96	cumulative	1.0	0.94	0.96

Another very important observation comes up from the comparison of the H@1 scores of HybEA-K and HybEA-K (struct. first) in Table 12 and the Recall results (since Precision is almost always 1.0) shown in the factual first and structural first parts of Table 8, respectively. We notice that the vast majority of matches identified by HybEA-K come from the reciprocity filter and only a very small number of matches are left to be decided by the ranking of the most likely matches returned by the last component.

In more detail, we observe the following:

- D-W (S): When executing the structural component first, the cumulative recall from the reciprocity filter is 0.85 and H@1 of HybEA-K (struct. first) is 0.86, i.e., a 1% total difference. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.82, but H@1 of HybEA-K is 0.92, i.e., a 10% total difference.
- D-W (D): When executing the structural component first, the cumulative recall from the reciprocity filter is 0.97 and H@1 of HybEA-K (struct. first) is also 0.97, i.e., the number of correct matches identified outside the reciprocity filter is negligible. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.95, but H@1 of HybEA-K is 0.98, i.e., a 2% total increase.
- D-W (SRPRS-N): When executing the structural component first, the cumulative recall from the reciprocity filter is 0.86 and H@1 of HybEA-K (struct.

first) is also 0.86, again with negligible contribution of non-reciprocity-filter suggested matches. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.83, but H@1 of HybEA-K is 0.90, i.e., a 7% increase.

- D-W (SRPRS-D): With the structural component first, the cumulative recall from the reciprocity filter is 0.93 and H@1 of HybEA-K (struct. first) is 0.93, once again a negligible contribution of the matches not provided by the reciprocity filter. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.92, but H@1 of HybEA-K is 0.96, i.e., a 4% difference.
- BBC-DB: The cumulative recall from the reciprocity filter when executing the structural component first is 0.98, while H@1 of HybEA-K (struct. first) is 0.98, once again showing no difference. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.95, but H@1 of HybEA-K is 0.99, i.e., an almost 4% total difference.
- ICEWS-WIKI: The cumulative recall obtained by the reciprocity filter when using the structural component first is 0.91, and H@1 of HybEA-K (struct. first) is 0.98. This is the first time we observe a significant difference between the recall of the reciprocity filter and the performance of the model when using the structural component first. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.91, while H@1 of HybEA-K is also 0.91. This is the first time we observe a negligible contribution between the recall of the reciprocity filter and the performance of the model when using the factual component first. It is also the first that we see the precision not be 1.0, but 0.99, proposing 1% wrong matching pairs.
- ICEWS-YAGO: The cumulative recall obtained by the reciprocity filter when using the structural component first is 0.94, and H@1 of HybEA-K (struct. first) is 0.95, i.e. a 1% difference. When executing the factual component first, the cumulative recall from the reciprocity filter is 0.94, while H@1 of HybEA-K is 0.94, i.e., once again a negligible contribution when using factual first. In addition, this dataset is another case where the precision is 0.99 instead of 1.0.

From the above findings, we can observe that the effectiveness of the reciprocity filter is on par with the results exhibited by HybEA-K: both are higher when using the factual component first, except from the datasets ICEWS-WIKI and ICEWS-YAGO. It is also interesting to note that the highest differences observed between the cumulative recall of the reciprocity filter and H@1 are on D-W (S) D-W (SRPRS-N) D-W (SRPRS-D) which are the datasets that showed the most important semantic heterogeneity in the dataset analysis section. It can thus be explained by the fact that one of the component has more difficulties to correctly align, leading to a significant difference between the reciprocity filter and the performance.

Table 9: Efficiency results of HybEA-K.

	Mean time per epoch (s)		Total number of epochs		Number of semi-supervision cycles	Total time factual (s)	Total time structural (s)	Total time (s)
	Factual	Structural	Factual	Structural				
D-W (S)	65.77	32.81	230	325	4	15,127	10,663	25,790
D-W (D)	59.74	62.70	220	205	4	13,142	12,853	25,996
D-W (SRPRS-N)	57.54	31.88	410	260	4	23,591	8,288	31,880
D-W (SRPRS-D)	53.55	57.77	300	235	4	16,065	13,575	29,640
BBC-DB	57.28	23.17	140	125	4	8,019	2,896	10,915
ICEWS-WIKI	4.05	1,455	30	15	1	122	21,836	21,958
ICEWS-YAGO	19.12	1,873	30	20	1	574	37,475	38,049

Table 10: Efficiency results of HybEA-R.

	Mean time per epoch (s)		Total number of epochs		Number of semi-supervision cycles	Total time factual (s)	Total time structural (s)	Total time (s)
	Factual	Structural	Factual	Structural				
D-W (S)	65.77	0.22	240	4,800	4	15,785	1,056	16,841
D-W (D)	59.74	0.30	200	4,800	4	11,948	1,440	13,388
D-W (SRPRS-N)	57.54	0.21	435	4,800	4	25,030	1,008	26,038
D-W (SRPRS-D)	53.55	0.28	250	4,800	4	13,388	1,344	14,732
BBC-DB	57.28	0.13	110	4,800	4	6,360	624	6,984
ICEWS-WIKI	4.05	1.12	30	1,200	1	122	1,742	1,864
ICEWS-YAGO	19.12	1.46	30	1,200	1	574	1,752	2,326

C.4 Efficiency analysis

In Tables 9 and 10, we present an efficiency analysis of both HybEA-K and HybEA-R building blocks respectively, providing the mean time per epoch for both factual and structural components, while also the total number of epochs needed by each component per dataset and the number of the semi-supervised cycles. In addition, apart from the total time that each component requires for training, we also provide the total training time that HybEA-R and HybEA-K needed for each dataset. Table 11 complements this analysis with the corresponding test-time cost of the factual and structural components.

Test time Table 11 reports the test time of the factual component and of the two structural backbones. Test time corresponds to a single final evaluation after training and includes embedding generation, similarity computation, and ranking. Both variants use the same factual architecture and inference procedure. Therefore, we report the factual test cost only once. The total test time ranges from 9.51 to 81.27s for HybEA-R and from 4.07 to 66.70s for HybEA-K, and remains substantially lower than the training cost reported in Tables 9 and 10.

Efficiency of building blocks We observe that despite the fact that the factual component brings huge improvement in our model (as we analyze later in Appendix C.5 and previously in Section 3.1), it is less efficient that the structural component in the vast majority of the datasets in both HybEA-R and HybEA-K (e.g., the factual component of HybEA-K in D-W (SRPRS-N) needs 23,591s, in contrast to the structural one which needs 8,288). This is not the case for the highly structural heterogeneous ICEWS-WIKI and ICEWS-YAGO burdening the structural component in terms of the mean time required per epoch, leading to higher total training time (e.g., 1,455s for each epoch and 21,836s total training time for ICEWS-WIKI and HybEA-K). Also, it is worth mentioning that the reciprocity filter (another building block of HybEA), despite the huge

Table 11: Test time (in seconds) of the factual and structural components.

Dataset	Factual	Structural (RREA)	Structural (Knowformer)
D-W (S)	50.32	15.95	8.77
D-W (D)	50.99	16.75	6.89
D-W (SRPRS-N)	59.83	21.44	6.87
D-W (SRPRS-D)	58.41	15.67	6.98
BBC-DB	34.27	13.90	2.94
ICEWS-WIKI	3.03	6.48	1.04
ICEWS-YAGO	21.47	42.07	12.95

improvement in terms of effectiveness that brings, it requires negligible time, that is near to zero.

Despite the large number of epochs that the structural component of HybEA-R (i.e., RREA(basic)) requires (1,200 per semi-supervised cycle), the total time it needs is less than the structural component of HybEA-K (i.e., Knowformer) with two order of magnitude less mean time per epoch (e.g., for D-W (S) the structural component of HybEA-R needs 0.22s, while the one of HybEA-K needs 32.81s). On the other hand, the higher effectiveness of RREA(basic) compared to Knowformer (as shown in Table 1), leads to less overhead in terms of time and number of epochs of HybEA-R factual model, compared to the one of HybEA-K in most datasets (e.g., the factual component of HybEA-R for D-W (D) needs 200 epochs and 11,948s, while the one of HybEA-K needs 220 epochs and 13,142s). D-W (S) and D-W (SRPRS-N) are the exception datasets, since they exhibit higher semantic heterogeneity, requiring bigger effort by the factual component. For ICEWS-WIKI and ICEWS-YAGO the training time that was required by the factual model remains the same for both HybEA-R and HybEA-K, since it only ran in one semi-supervised cycle with negligible number of newly generated matching pairs from the reciprocity filter.

Efficiency per dataset In both tables, we observe that while most of the datasets need four semi-supervision cycles to train, ICEWS-WIKI and ICEWS-YAGO need only one, since the newly generated matching pairs are limited, consequently the low number of total epochs in both HybEA-R and HybEA-K and training time needed by each component individually and in total in HybEA-R. In addition, since BBC-DB is a dataset with both low factual and structural heterogeneity compared to the other datasets, it needs lower time in total, but also for each component individually, for both HybEA-R and HybEA-K. Last but not least, we observe that the dense versions D-W (D) and D-W (SRPRS-D) of D-W (S) and D-W (SRPRS-N), require more effort by the structural component due to the more structural information, leading to higher individual and total training time in both HybEA-R and HybEA-K, compared to the sparse versions.

Efficiency per method HybEA-R is more efficient than HybEA-K in terms of the total time required for training, while also it outperforms it in almost all the datasets in terms of effectiveness (see Table 1). This is due to the high efficient

and performant structural component of HybEA-R(RREA), that outperforms Knowformer of HybEA-K in both efficiency and effectiveness.

Table 12: Ablation study for HybEA-K.

Method	Metric	D-W (S)	D-W (D)	D-W (SRPRS-N)	D-W (SRPRS-D)	BBC-DB	ICEWS-WIKI	ICEWS-YAGO
HybEA-K	H@1	0.920	0.988	0.908	0.968	0.996	0.916	0.941
	H@10	0.969	0.997	0.954	0.987	0.997	0.938	0.944
	MRR	0.93	0.99	0.92	0.97	0.99	0.92	0.94
HybEA-K (basic)	H@1	0.877	0.970	0.850	0.935	0.992	0.916	0.941
	H@10	0.962	0.995	0.936	0.981	0.996	0.938	0.944
	MRR	0.90	0.97	0.88	0.95	0.99	0.92	0.94
HybEA-K (basic; str. first)	H@1	0.820	0.947	0.827	0.913	0.977	0.985	0.952
	H@10	0.837	0.955	0.855	0.919	0.978	0.995	0.976
	MRR	0.83	0.95	0.84	0.91	0.98	0.98	0.96
HybEA-K (struct. first)	H@1	0.861	0.971	0.863	0.938	0.984	0.985	0.952
	H@10	0.872	0.974	0.881	0.942	0.985	0.995	0.976
	MRR	0.86	0.97	0.87	0.94	0.99	0.98	0.96
HybEA-K (w/o fact.)	H@1	0.753	0.917	0.603	0.873	0.974	0.073	0.019
	H@10	0.936	0.987	0.815	0.970	0.995	0.275	0.095
	MRR	0.82	0.94	0.67	0.90	0.98	0.13	0.04
HybEA-K (w/o struct.)	H@1	0.558	0.621	0.513	0.444	0.748	0.926	0.948
	H@10	0.612	0.672	0.592	0.488	0.769	0.974	0.975
	MRR	0.79	0.82	0.70	0.70	0.91	0.94	0.96

Table 13: Ablation study on HybEA-R initialization.

Method	Metric	D-W (S)	D-W (D)	D-W (SRPRS-N)	D-W (SRPRS-D)	BBC-DB	ICEWS-WIKI	ICEWS-YAGO
HybEA-R	H@1	0.989	1.000	0.972	0.997	0.993	0.994	0.994
with random initialization	H@10	0.997	1.000	0.992	1.000	1.000	0.997	0.997
	MRR	0.99	1.00	0.98	1.00	1.00	1.00	1.00
HybEA-R	H@1	0.990	0.999	0.974	0.996	0.994	0.994	0.994
with SentenceBert initialization	H@10	0.998	1.000	0.993	1.000	1.000	0.997	0.998
	MRR	0.99	1.00	0.98	1.00	1.00	0.99	1.00

C.5 Ablation study of HybEA-K components

We conduct an ablation study, showing that by changing any single component of HybEA-K, the entire performance drops (Table 12). This demonstrates that all of our architectural choices have a positive impact on the effectiveness of HybEA-K.

Impact of component ordering. We experiment with different orders of execution for the structural and factual components, as presented in row HybEA-K (struct. first) of Table 12. We observe that we obtain different results, with winner in the vast majority of the datasets, being running the factual component first (HybEA-K), that was finally chosen as our default configuration.

Impact of the semi-supervision step. Table 12 reports the results of our method at the end of the first cycle, when running the structural component first, denoted as HybEA-K (basic; str.first), as well as when running the factual component first, denoted as HybEA-K (basic). Compared to the results of the full pipeline, with the corresponding order of components, the basic version always yields lower or equivalent results. The difference between the basic version of

Table 14: Efficiency of embedding-based EA methods with training time(in seconds).

Dataset	HybEA-R	HybEA-R(basic)	RREA(basic)	Knowformer	SelfKG
D-W (S)	16,841	5,526	264	3,298	10,936
D-W (D)	13,388	3,646	360	3,593	10,970
D-W (SRPRS-N)	26,038	5,718	252	2,567	11,087
D-W (SRPRS-D)	14,732	5,423	336	3,360	10,972
BBC-DB	6,984	1,015	156	1,004	6,909
ICEWS-WIKI	1,864	1,405	1,344	29,114	10,175
ICEWS-YAGO	2,326	2,039	1,752	46,844	17,291

HybEA-K and the full version of HybEA-K is bigger than the corresponding of Table 4. The difference between the basic and the full version is similar when comparing the results of HybEA-K (basic; str. first) with HybEA-K (struct. first). It is worth noticing that the basic version of HybEA-K already outperforms all the state-of-the-art baseline methods.

Impact of the factual component. The row HybEA-K (w/o fact.) skips the factual component and essentially runs the structural component of HybEA-K only (i.e., Knowformer) using SentenceBERT initialization and enriched with the reciprocity filter. This experiment involves only one cycle, since re-training the same model without new pairs suggested from the other model, yields the same results in subsequent cycles. Although the reciprocity filter results show perfect precision in almost all cases (in Table 8), the results of HybEA-K (w/o fact.) in Table 12 are not identical with the results of Knowformer in Table 1, since Knowformer does not use SentenceBERT initialization. Instead, HybEA-K (w/o fact.) outperforms Knowformer, showing that the SentenceBERT initialization of the entity embeddings brings big improvement. We observe that *the factual component has a huge impact* on the results, since, when we remove it, the H@1 scores drop by 2.2% to even 84.6%.

Impact of the structural component. Similarly, HybEA-K (w/o struct.) presents the results of our method without the structural component, i.e., it includes the execution of the factual component only, in one cycle. Despite the fact that running the factual component first yields better results than the reverse order, yet, running the factual component alone, yields low-quality results; usually worse than running the structural component only. This observation does not hold for the ICEWS-WIKI and ICEWS-YAGO datasets, though, for the reasons described earlier, in Section B.3. This means that by running the factual component first, we exploit the two components better, compared to running the structural component first.

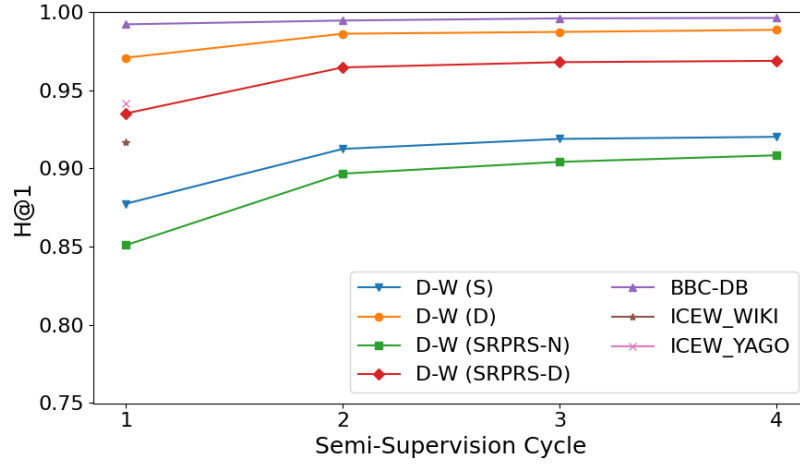
C.6 Ablation study on HybEA-R initialization

We analyze the influence of different types of entity initialization strategies on HybEA-R performance. Specifically, we compare random initialization with Sentence-BERT initialization. As shown in Table 13, the results reveal that the

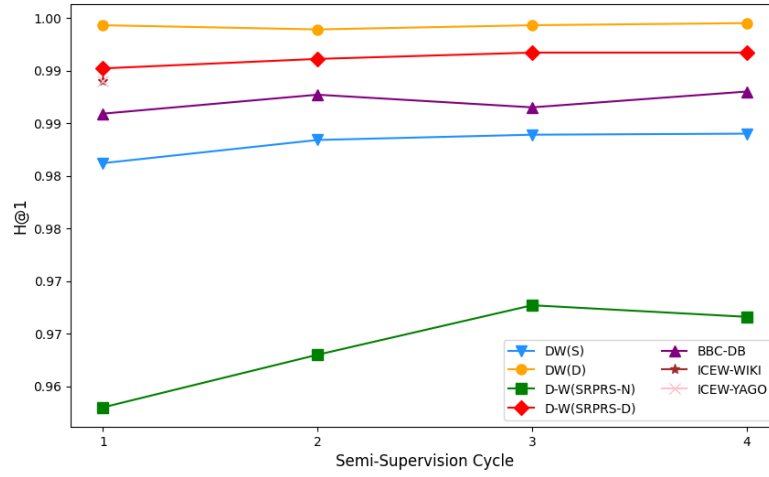
choice of initialization has little to no impact on the performance of HybEA-R. The H@1, H@10, and MRR scores remain almost identical across all datasets, with only negligible variations. The largest difference is observed on the D-W (SRPRS-N) dataset, where there is a 0.2% difference in H@1. This proves that HybEA-R efficiently learns robust entity embeddings, making it less dependent on pre-trained embeddings. Given that HybEA-R already captures most alignment cues effectively, the additional information from Sentence-BERT initialization has only small impact on performance. Based on these results, we opted to keep random initialization for HybEA-R, as it gives the same high performance while avoiding the extra computational cost of Sentence-BERT embeddings.

C.7 Efficiency and Effectiveness Trade-off

Table 14 reports training cost and complements Table 1. Because HybEA-R runs up to four semi-supervised cycles, its training time can be up to approximately four times longer than HybEA-R (basic). Early stopping often reduces the number of executed cycles, especially on FR-EN and JA-EN, so observed ratios are frequently much smaller. HybEA-R uses RREA as its structural backbone, so the additional cost mainly comes from the factual component and the co-training loop, which are also key to its improved performance. Lightweight structural baselines such as RREA (*basic*) are extremely fast but, depending on the dataset, may lose up to ~ 10 points in H@1. On ICEWS datasets several baselines both suffer large drops in H@1 and train longer. Overall, HybEA-R stays under about 2,300 seconds in our runs and approaches 1.0 H@1 on those datasets. When time is limited, HybEA-R (*basic*) can reduce training time with a modest accuracy drop that remains above the state of the art on several datasets. Alternatively one can lower the maximum number of cycles to further trade speed for accuracy.



(a) HybEA-K



(b) HybEA-R.

Fig. 5: Comparison of performance per cycle.